



지원 직무 : AI 서비스 백엔드 개발자

이메일 020799@naver.com 연락처 010-3379-3950
포트폴리오 [jw020799.github.io](https://github.com/jw020799) GitHub github.com/jw020799
주소 경기도 고양시

핵심 요약 Executive Summary

FastAPI · Python 기반의 AI 서비스 백엔드 개발자입니다. 금융 IT 회사에서 2년간 보안문서를 분류하고 디지털화하며 문서를 찾고 검증하는 일을 반복했고, 그 과정을 자동화하고 싶어 개발로 전환했습니다.

AI 결과를 그대로 내보내지 않고 서비스 기준 안에서 검증하는 데 강점이 있습니다. 근거가 부족한 리포트는 FAILED로 분리했고, 재무 데이터 수집이 실패하면 unavailable로 반환했으며, 검색 근거는 화면에 그대로 노출했습니다.

문제가 생기면 원인을 지표로 확인하고, 좋은 결과보다 실패 조건을 먼저 정의합니다.

기술 역량 Technical Skills

Language	Python, JavaScript Python으로 FastAPI 백엔드와 AI 서비스 로직을, JavaScript(React)로 화면을 구현했습니다.
Backend	FastAPI, SQLAlchemy, Pydantic, REST API GIUT에서 Event / Course 모델 · 스키마와 API를, TUKTAK에서 Risk Report API 구조를 설계 · 구현했습니다.
Database · Vector DB	MySQL, SQLite, ChromaDB MySQL 기반 행사 · 사용자 데이터 모델링, ChromaDB 컬렉션 구성과 메타데이터 필터 검색을 구현했습니다.
AI · RAG	LangChain, LangGraph, OpenAI API, Hugging Face, Sentence-Transformers RAG 리스크 문서 검색 흐름을 설계 · 검증하고, LangGraph로 Agent 6개 워크플로우를 구성했습니다.
AI Evaluation	Ko-SRoBERTa, Nomic Embed Vision, Cosine Similarity, Threshold 설계 임베딩 모델을 비교하고 카테고리별 Threshold를 설계해 오답 통과 0개 기준을 충족했습니다.
Frontend	React, Vite, Figma TUKTAK 고객 · 시공사 마이페이지와 홈 화면, GIUT 주요 화면을 구현하고 백엔드 API를 연동했습니다.
DevOps · Collaboration	Docker, Docker Compose, GitHub Actions, AWS EC2, Git / GitHub (Branch · PR), Monday Docker 실행 환경과 GitHub Actions Docker build 워크플로우를 구성하고, EC2 배포와 Branch / PR 협업을 경험했습니다.



TUKTAK : AI 기반 집수리 견적 · 리스크 리포트 · 시공사 역경매 매칭 플랫폼

사진과 설명을 입력하면 AI 견적서와 리스크 리포트를 생성하고 시공사 매칭으로 이어지는 서비스

기간 2026.06.12 ~ 2026.08.04 | 4명 | AI 백엔드 · RAG · 이미지 유사도 검색 담당
기술 스택 Python, FastAPI, SQLAlchemy, MySQL, LangChain, LangGraph, ChromaDB, Hugging Face, OpenAI API, React, Docker, AWS, GitHub Actions

담당 역할

- Ko-SRoBERTa + ChromaDB 기반 RAG 리스크 문서 검색 설계 · 구현 (문서 71개, 리스크 유형별 Threshold, 메타데이터 필터)
- Risk Report 도메인 모델 및 API 구조 구현 (Main Backend)
- React 기반 고객 · 시공사 마이페이지 · 홈 화면 구현, AI 리스크 페이지 연동, API 응답 실패 시 fallback 처리
- 이미지 유사사례 검색 모델 비교(DINOv2 · Nomic Embed Vision · CLIP · ConvNeXt) 및 카테고리별 Threshold 설계
- 수리 요청 구조화 모델(KLUE-RoBERTa-base) 성능 검증, AI Service Dockerfile · GitHub Actions 빌드 구성

성과 및 정량적 지표

- 수리 요청 구조화: KLUE-RoBERTa-base로 평균 정확도 86.4% 확보. 요청당 0.014초로 LLM 호출 없이 처리해 응답 지연과 API 비용을 함께 줄임
- 임베딩 모델 4종(DINOv2 · Nomic Embed Vision · CLIP · ConvNeXt)을 같은 기준으로 비교해 최종 모델과 카테고리별 Threshold 선정
- 이미지 유사사례 검색: 잘못된 사례가 견적 근거로 쓰이는 위험을 0으로 두는 것을 우선해 오답 통과 0개 기준으로 Threshold 설계. 이 기준에서 정답 통과율은 60.4%이며, 걸러진 애매한 사례는 LLM 판단으로 분기해 보완
- RAG 리스크 리포트 카테고리 10개 중 근거 충분 9개는 COMPLETED, 근거 부족 1개는 FAILED로 분리해 잘못된 위험도 노출을 차단

트러블슈팅 · 문제 해결 경험

[근거 부족 케이스가 낮은 위험도로 출력되는 문제]

- [문제] 근거 문서가 부족한 카테고리(예: 가전, 근거 2/5)가 낮은 위험도로 정상 출력되고, 통계 자료가 가격 근거로 통과됨
- [원인 분석] 유사도 점수만으로 결과를 확정하고, 리스크 항목별 근거 수와 문서 유형을 확인하지 않음
- [해결] 검색을 리스크 항목 5개(PRICE · EXTRA_COST · SAFETY · CONTRACT · FIELD)로 나누고 문서 유형 · 카테고리 메타데이터 필터 적용, UNKNOWN 3개 이상이면 FAILED 처리
- [결과] 근거 부족 카테고리가 FAILED로 분리되어 잘못된 위험도 노출 차단. 좋은 결과보다 실패 조건을 먼저 정의하는 습관을 얻음

[이미지 유사사례 검색 기준에 대한 팀 의견 차이]

- [문제] 유사사례를 최대한 많이 찾자는 의견과 오답 통과를 엄격히 막자는 의견이 갈림
- [원인 분석] 정확도 단일 지표로 평가해, 잘못된 사례가 견적 기준으로 쓰일 위험과 카테고리별 난이도 차이를 반영하지 못함
- [해결] 오답 통과율을 우선 지표로 바꾸고 카테고리별 보수적 Threshold 적용, 애매한 경우는 LLM 판단으로 분기
- [결과] 오답 통과 0개, 정답 통과율 60.4% 기준으로 팀 합의. 의견 차이를 테스트 결과 기준으로 정리하는 방법을 배움



기간 2026.08.05 ~ 2026.08.10 | 1명 | 전체 설계 · 구현

기술 스택 Python 3.11, FastAPI, SQLAlchemy, LangGraph, ChromaDB, Sentence-Transformers, Streamlit, yfinance, Google News RSS, Docker Compose

담당 역할

- Google News RSS 뉴스 수집, yfinance 재무 데이터 수집 및 실패 처리 구현
- Sentence-Transformers 임베딩 + ChromaDB RAG 검색 구조 구현
- LangGraph 기반 Planner · RAG · News · Finance · Risk · Report Agent 워크플로우 구성 (Rule-based baseline, LLM Agent로 교체 가능한 구조)
- FastAPI 리포트 생성 / 저장 / 조회 API, SQLAlchemy 저장 구조, Streamlit 데모 화면, Docker Compose 실행 환경 구성

성과 및 정량적 지표

- 수집 → 임베딩 → 검색 → Agent 분석 → 리포트 → API까지 전체 흐름을 부트캠프 수료 다음 날부터 6일 만에 단독 구현
- Agent 6개, API 엔드포인트 4개, Docker Compose로 API · UI 두 서비스 동시 실행
- README에 아키텍처 · API 명세 · 트러블슈팅 문서화

트러블슈팅 · 문제 해결 경험

[yfinance 요청 제한(429)으로 리포트 생성이 중단되는 문제]

[문제] 재무 데이터 수집 중 429 Too Many Requests 오류가 나면 리포트 생성 전체가 중단

[원인 분석] stock.info 등 부하가 큰 요청을 반복해 Yahoo Finance 요청 제한에 걸림

[해결] fast_info · 최근 가격 이력을 우선 사용하고, 실패 시 **status: unavailable**로 반환하는 실패 처리 추가

[결과] 재무 데이터가 없어도 뉴스 · RAG · Agent 분석으로 리포트 생성 유지, 실패 원인을 응답에 그대로 노출

[RAG 검색 결과에 유사도 낮은 문서가 섞이는 문제]

[문제] 검색된 뉴스 중 관련성이 낮은 문서가 리포트 근거로 들어갈 수 있음

[원인 분석] 경량 임베딩 모델을 쓰고, 최근 수집한 소량의 뉴스만 대상으로 검색해 유사도 분포가 낮음

[해결] 유사도 점수와 참조 문서를 UI · API 응답에 그대로 노출해 근거를 검토할 수 있게 하고, 임베딩 모델 비교 · Threshold 필터링 · 뉴스 중복 제거를 확장 과제로 문서화

[결과] 근거가 약한 리포트를 사용자가 바로 판별 가능. TUKTAK에서 배운 "근거 부족은 숨기지 말고 드러낸다"는 기준을 개인 프로젝트에도 적용



기간 2026.04.13 ~ 2026.05.27 | 5명 | 백엔드 모델 · API 및 화면 일부 담당
기술 스택 Python, FastAPI, MySQL, React, OpenAI API, 한국관광공사 API, Naver Map API, Docker Compose, AWS EC2

담당 역할

- Event · EventDetail · Course 모델 및 Pydantic 스키마 설계, Course API 구현
- FastAPI 기반 회원가입 / 로그인 API, 지역 · 날짜 · 키워드 조건 행사 검색 · 필터링 API 구현
- React 주요 화면 구현 및 백엔드 API 응답 연동

성과 및 정량적 지표

- 외부 API 응답 구조를 그대로 옮긴 데이터 모델을 서비스 요구 기준으로 재설계: 컬럼 19개 → 10개, 화면에서 쓰지 않는 필드 제거로 모델 · 스키마 · 화면 불일치 해소
- 회원가입 · 로그인부터 지역 · 날짜 · 키워드 조건 검색까지 API 전 구간 구현
- 공공데이터 · OpenAI API · MySQL을 연동한 풀스택 서비스를 AWS EC2에 Docker Compose로 배포
- 5명 팀에서 백엔드 모델 · API를 담당하며 Git Branch · PR 기반 협업 경험

트러블슈팅 · 문제 해결 경험

[공공데이터 응답 구조를 그대로 옮긴 Event 모델]

- [문제] 컬럼이 19개로 늘고 모든 컬럼에 event_ 접두어가 붙어 Pydantic 스키마와 이름이 어긋나며, 화면에서 쓰지 않는 필드까지 관리
- [원인 분석] 한국관광공사 API 응답 필드(contentid, mapx / mapy, lclsSystem3 등)를 서비스 요구와 무관하게 그대로 옮김
- [해결] 화면에 필요한 필드(제목 · 지역 · 기간 · 장소 · 이미지 · 설명 · 좋아요 수) 중심으로 재설계하고 접두어 제거, EventDetail · Course도 같은 기준으로 정리
- [결과] 컬럼 19개 → 10개, 모델 · 스키마 필드명 일치. 외부 API 구조가 아니라 서비스 요구에서 모델을 시작해야 한다는 점을 배움

교육 및 활동 Education & Experience

IBM x Red Hat AI 부트캠프 2026.02 ~ 2026.08 | 6개월

IBM x Red Hat

- Python · FastAPI · React · MySQL · Docker · AWS 기반 웹 서비스 개발 과정 이수
- 중간 팀 프로젝트 GIUT(백엔드 모델 · API), 최종 팀 프로젝트 TUKTAK(AI 백엔드 · RAG) 수행

경민대학교 2018.03 ~ 2021.02

호텔관광경영과 | 졸업